252-0027

Einführung in die Programmierung

Übungen

Woche 6: References, Recursion, Precondition

Joran Bühler
Departement Informatik
ETH Zürich

Nachbesprechung

```java
public static boolean fast12(int z)
    int num = z % 12;
    if (num < 3 || num > 9) {
        return true;
    }
    else {
        return false;
    }
}
```

```java
public static boolean fast12(int z) {
    int num = z%12;
    return num < 3 || num > 9;
}
```

Bonus

1. `boolean containsSubstringAt(String str, int position, String sub)`: Diese Methode soll überprüfen, ob `sub` im String `str` an der Position `position` vorkommt. Zum Beispiel:

   - `containsSubstringAt("abcd", 0, "ab"): true`
   - `containsSubstringAt("abcd", 1, "ab"): false`
   - `containsSubstringAt("abcd", 4, "ab"): false`
   - `containsSubstringAt("abcd", -1, "ab"): false`

   Hinweis: Achten Sie darauf, dass die Methode auch `false` zurückgeben soll, wenn `position` kein valider Index des Strings `str` ist (also wenn `position` negativ oder zu gross ist).

2. `int countSubstrings(String str, String sub)`: Diese Methode soll zählen, wie oft der String `sub` als Substring in `str` vorkommt - die Substrings können sich hier überlappen. Zum Beispiel:

   - `countSubstrings("aaa", "aa"): 2`
   - `countSubstrings("aaaa", "aa"): 3`
   - `countSubstrings("12341234123", "123412"): 2`

Bonus

3. `int countDisjointSubstrings(String str, String sub)`: Diese Methode soll zählen, wie oft `sub` *ohne Überlappungen* in `str` vorkommt. Das erste Vorkommen (von links) eines Substrings `sub` trägt also zum Rückgabewert der Methode bei. Jedes weitere Vorkommen trägt nur zum Rückgabewert bei, wenn sich dieses nicht mit einem beitragenden Vorkommen weiter links in `str` überschneidet. Zum Beispiel:

   - `countDisjointSubstrings("abcdcba", "abc"):  1`
   - `countDisjointSubstrings("abcdcba", "c"):  2`
   - `countDisjointSubstrings("0101010", "010"):  2`

Hinweis: Sie können bei Aufgabe 6 annehmen, dass weder `str` noch `sub` eine Null-Referenz oder den leeren String enthalten.

My solution:

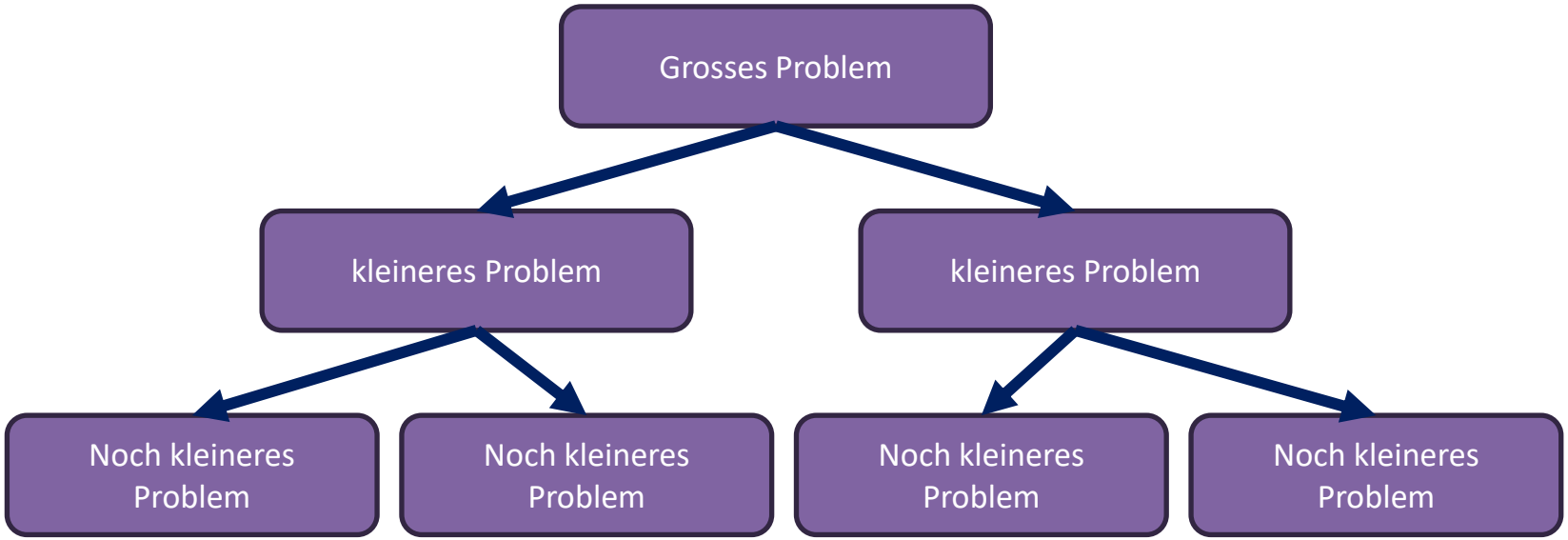
```java
public static boolean containsSubstringAt(String str, int position, String sub) {  7 usages
    //base case: check for bounds
    if (position + sub.length() > str.length() || position >= str.length() || position < 0) {
        return false;
    }
    //return true if substring at position is equal
    return str.substring(position, position + sub.length()).equals(sub);
}

public static int countSubstrings(String str, String sub) {  4 usages
    int counter = 0;
    //check at each index if the substring is contained
    for(int i=0; i <= str.length()-sub.length(); i++){
        if(containsSubstringAt(str, i, sub)){
            //found substring at position i
            counter ++;
        }
    }
    return counter;
}
```

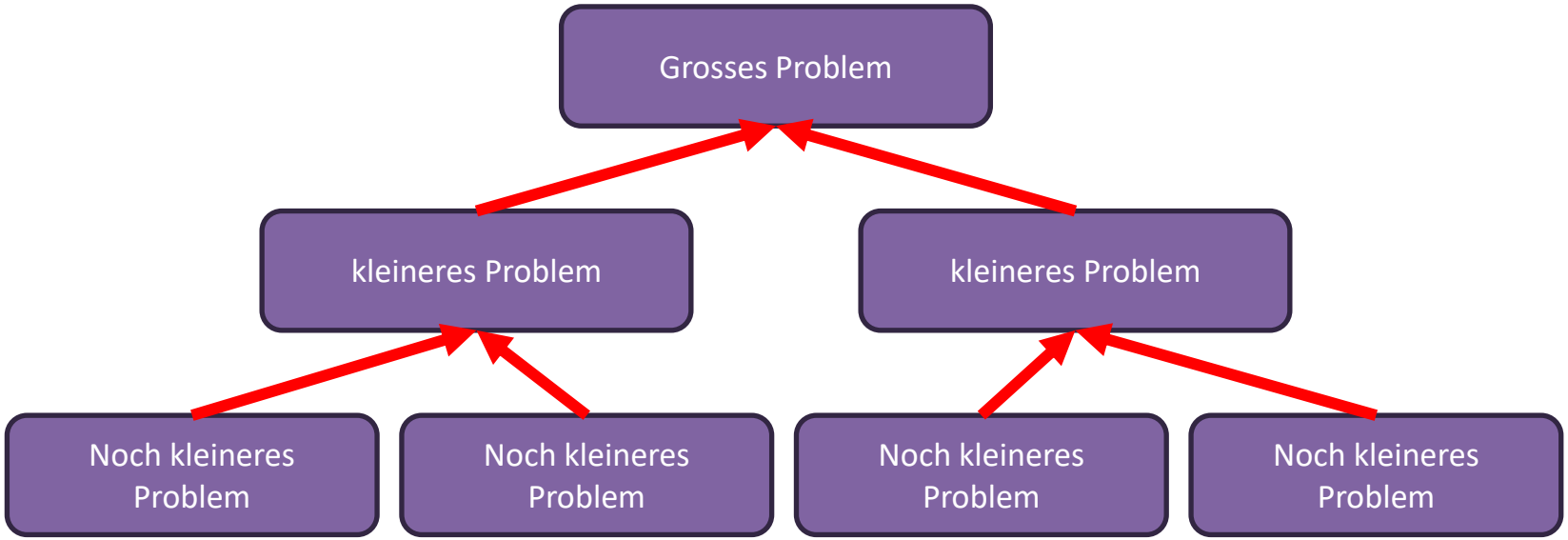
```java
//should throw an error if substring.length() == 0
public static int countDisjointSubstrings(String str, String sub) {  4 usages
    int counter = 0;
    //note that I don't increase i in the for loop itself
    for(int i=0; i <= str.length()-sub.length();){
        if(containsSubstringAt(str, i, sub)){
            counter ++;
            i+=sub.length();
        }
        else i++;
    }
    return  counter;
}
```

Rekursion

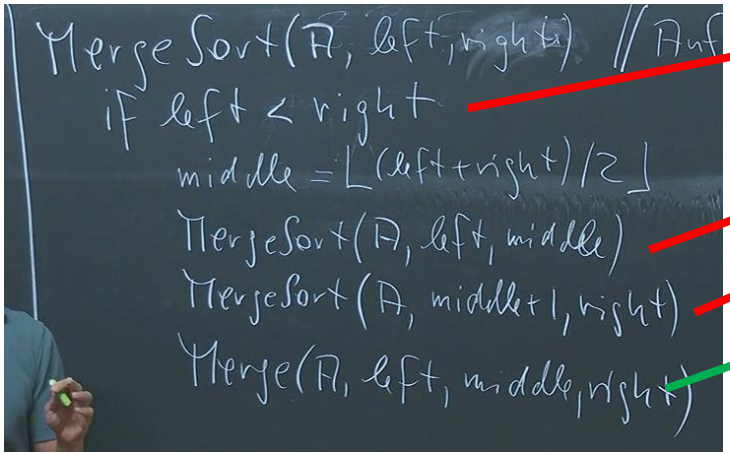
Rekusion – Grundprinzip Divide and Conquer

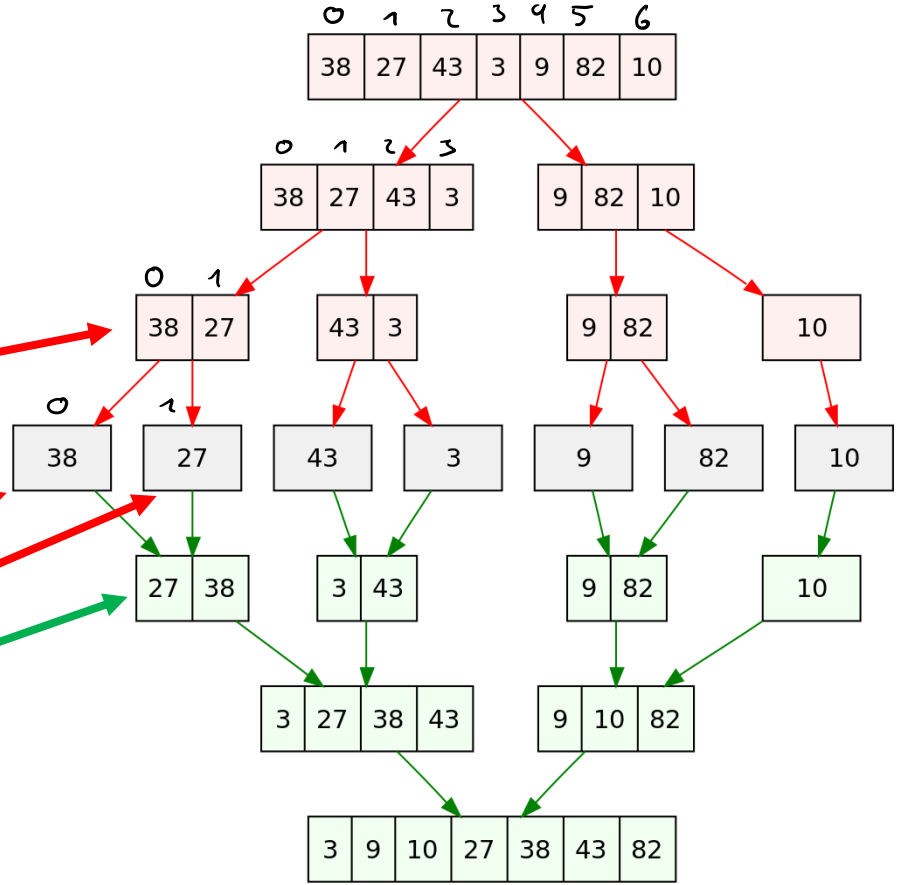


Rekusion – Grundprinzip Divide and Conquer

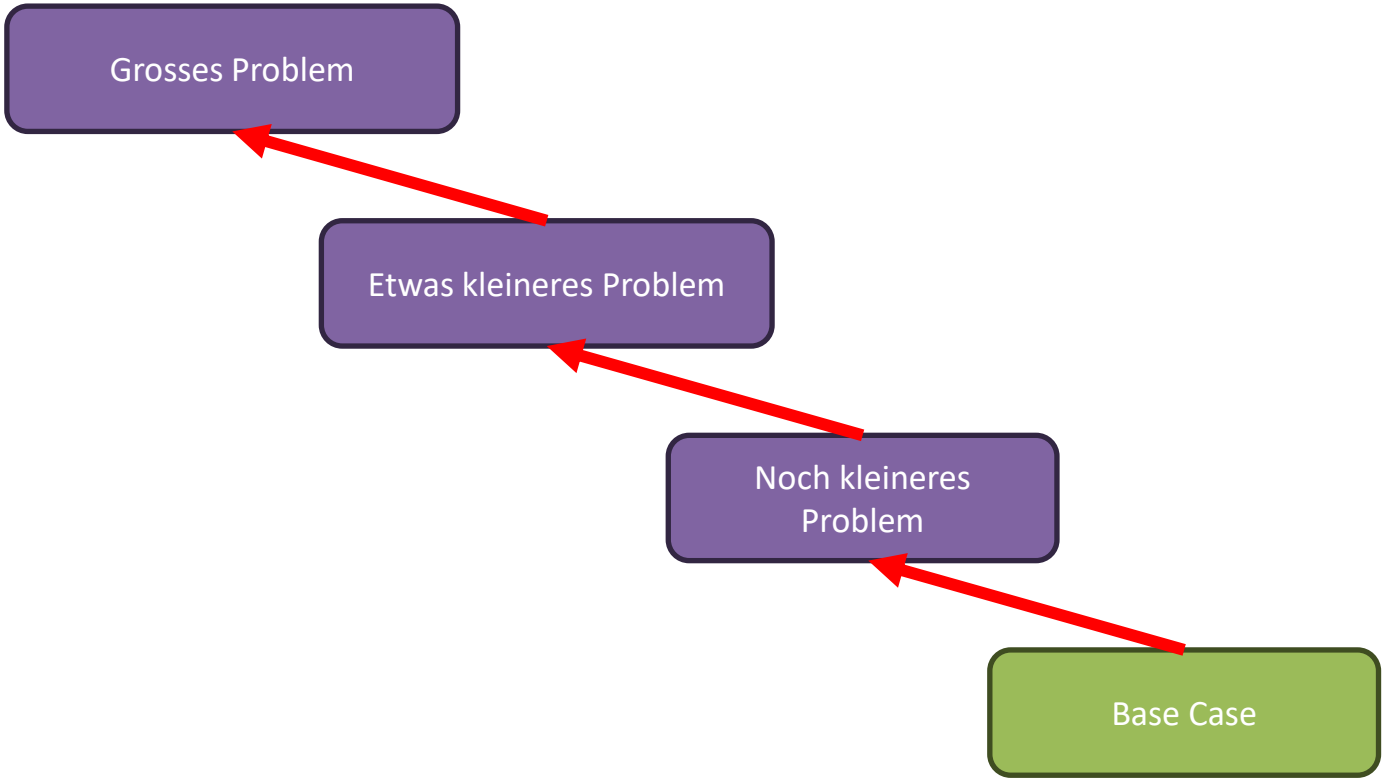


Beispiel: Mergesort





Rekusion – Decrease and Conquer



Decrease-and-Conquer - Fakultät

Überlegt euch: Wie berechnet man n! rekursiv?
Was ist der Base Case?
Was ist der Rekursive Fall?

Beispiel: 4! = 4 * 3 * 2 * 1 = 24

Decrease-and-Conquer - Fakultät

```
public int factorial(int n) {
    if (n==1)return 1;
    return n*factorial(n-1);
}
```

factorial(3)

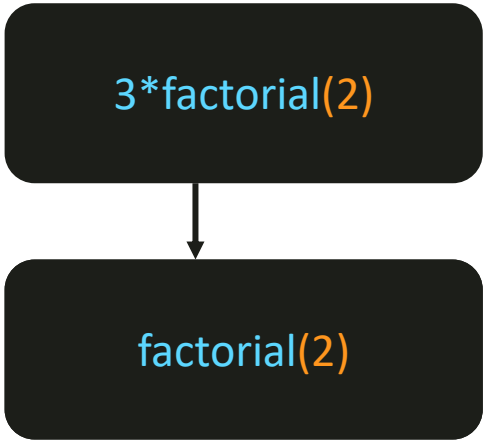
Decrease-and-Conquer - Fakultät

```
public int factorial(int n) {
    if (n==1)return 1;
    return n*factorial(n-1);
}
```

3*factorial(2)

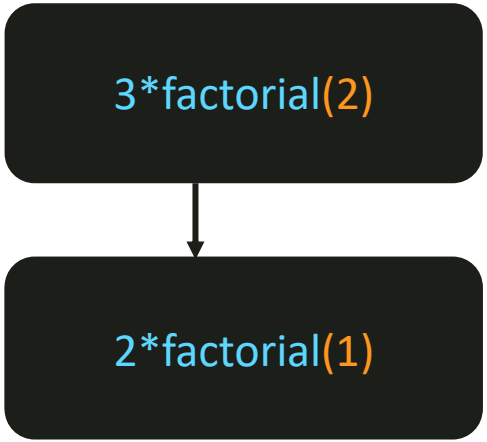
Decrease-and-Conquer - Fakultät

```java
public int factorial(int n) {
    if (n==1)return 1;
    return n*factorial(n-1);
}
```



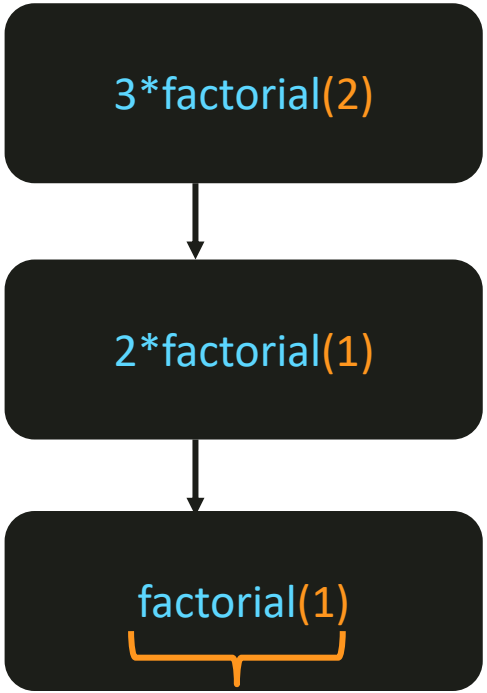
Decrease-and-Conquer - Fakultät

```java
public int factorial(int n) {
    if (n==1)return 1;
    return n*factorial(n-1);
}
```



Decrease-and-Conquer - Fakultät

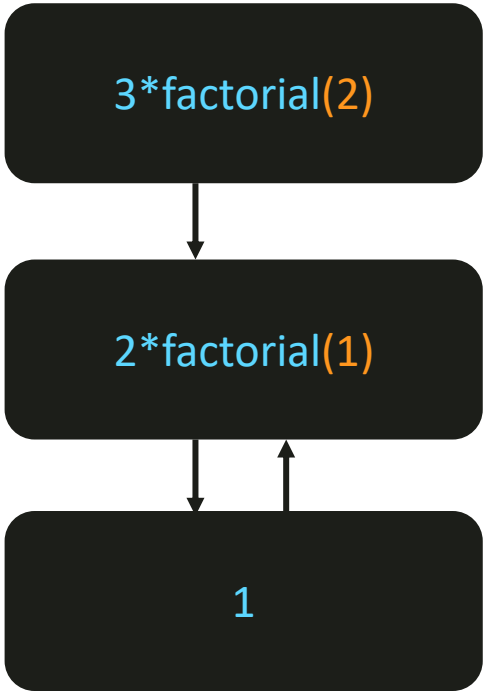
```java
public int factorial(int n) {
    if (n==1)return 1;
    return n*factorial(n-1);
}
```



Base Case!

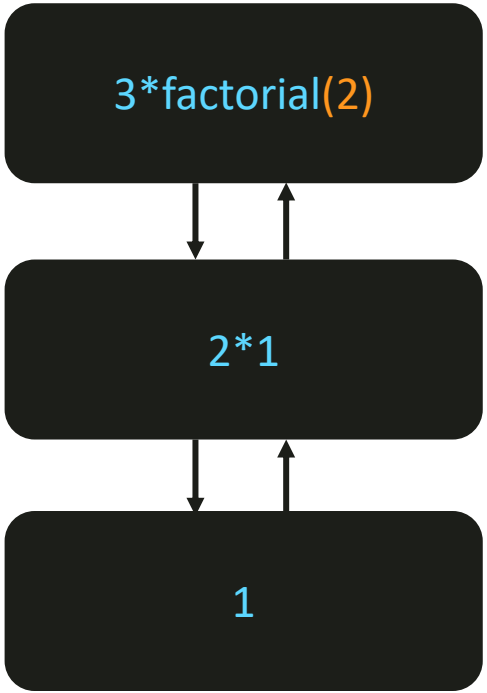
Decrease-and-Conquer - Fakultät

```java
public int factorial(int n) {
    if (n==1)return 1;
    return n*factorial(n-1);
}
```



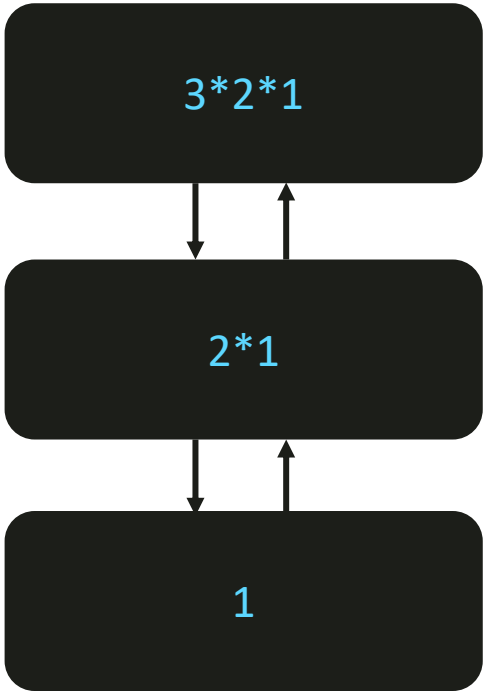
Decrease-and-Conquer - Fakultät

```java
public int factorial(int n) {
    if (n==1)return 1;
    return n*factorial(n-1);
}
```



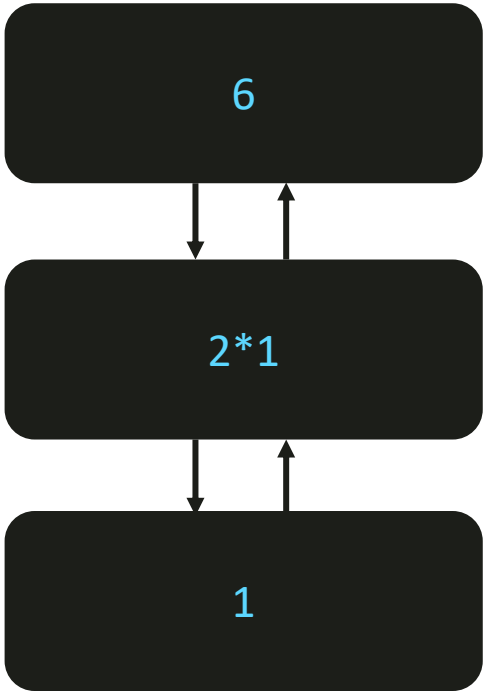
Decrease-and-Conquer - Fakultät

```java
public int factorial(int n) {
    if (n==1)return 1;
    return n*factorial(n-1);
}
```



Decrease-and-Conquer - Fakultät

```java
public int factorial(int n) {
    if (n==1)return 1;
    return n*factorial(n-1);
}
```



Rekursion – Step by Step

Problem, das wir lösen wollen

Aufgabe: Schreiben Sie eine Funktion `sum(n)`, die gegeben eine nichtnegative Zahl, alle nichtnegativen Zahlen bis einschliesslich n aufsummiert.

Beispiele:

sum(0) = 0

sum(1) = 1 = 1 + 0

sum(4) = 10 = 4 + 3 + 2 + 1 + 0

Wir wollen dies nun **rekursiv** lösen.

Schritte

1. Was ist die einfachste mögliche Eingabe?
2. Beispiele ausprobieren und visualisieren
3. Leite größere Beispiele von kleineren Beispielen ab
4. Verallgemeinere das Muster
5. Schreibe den Code

Schritte

1. Was ist die einfachste mögliche Eingabe?

   → Base Case

2. Beispiele ausprobieren und visualisieren
3. Leite größere Beispiele von kleineren Beispielen ab
4. Verallgemeinere das Muster

   → Rekursionsschritt

5. Schreibe den Code

1. Die einfachste Eingabe

- Hier ist es **n = _ : sum (n) = __**
- Die einfachste Eingabe ist später oft unser *Base Case*.
- Der Base Case ist der einzige Fall einer rekursiven Funktion, in dem wir eine direkte Lösung angeben.
- Eine Funktion kann auch mehrere Base Cases haben.
- Alle andere Lösungen bauen auf dem Base Case auf.

1. Die einfachste Eingabe

- Hier ist es **n = 0: sum(0) = 0**
- Die einfachste Eingabe ist später oft unser *Base Case*.
- Der Base Case ist der einzige Fall einer rekursiven Funktion, in dem wir eine direkte Lösung angeben.
- Eine Funktion kann auch mehrere Base Cases haben.
- Alle andere Lösungen bauen auf dem Base Case auf.

Schritte

1. Was ist die einfachste mögliche Eingabe?

→ Base Case



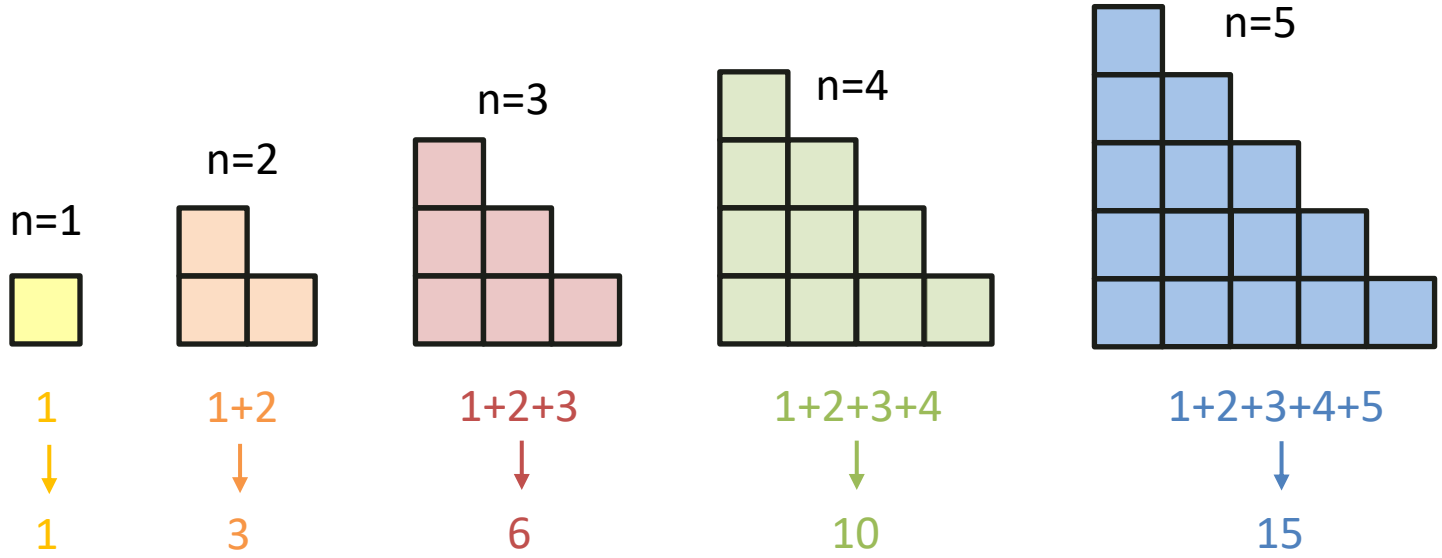
2. Beispiele ausprobieren und visualisieren
3. Leite größere Beispiele von kleineren Beispielen ab
4. Verallgemeinere das Muster

→ Rekursionsschritt

5. Schreibe den Code

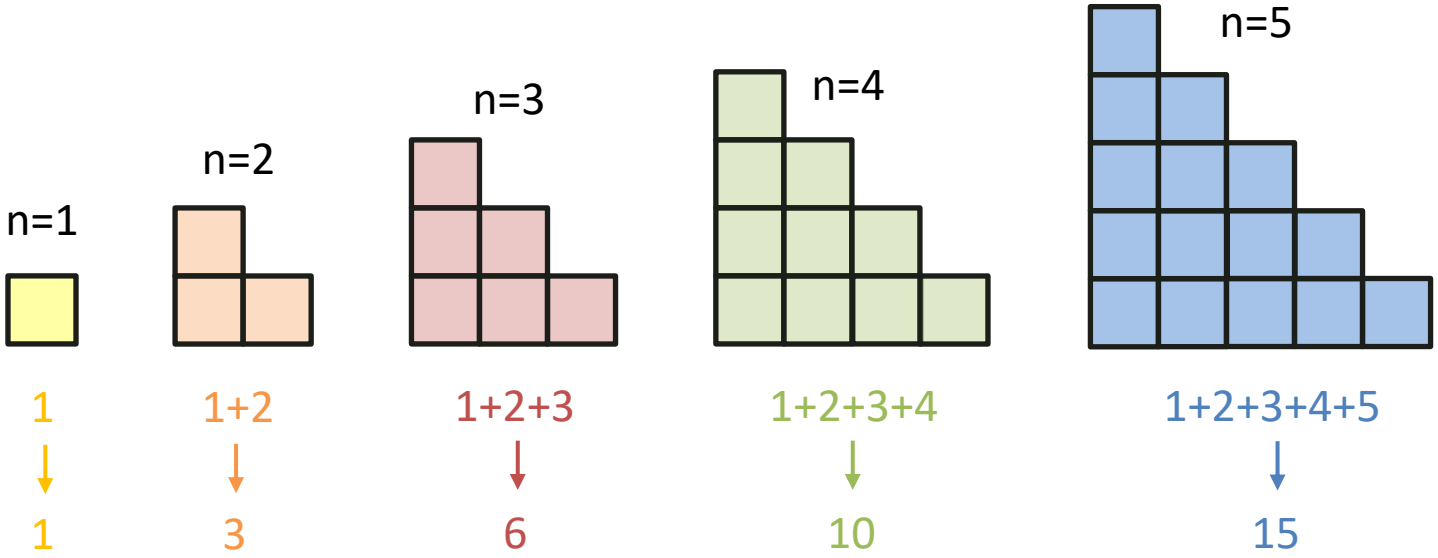
2. Die Beispiele

- Visualisiere, wie die Ein- und Ausgaben der Funktion miteinander zusammenhängen



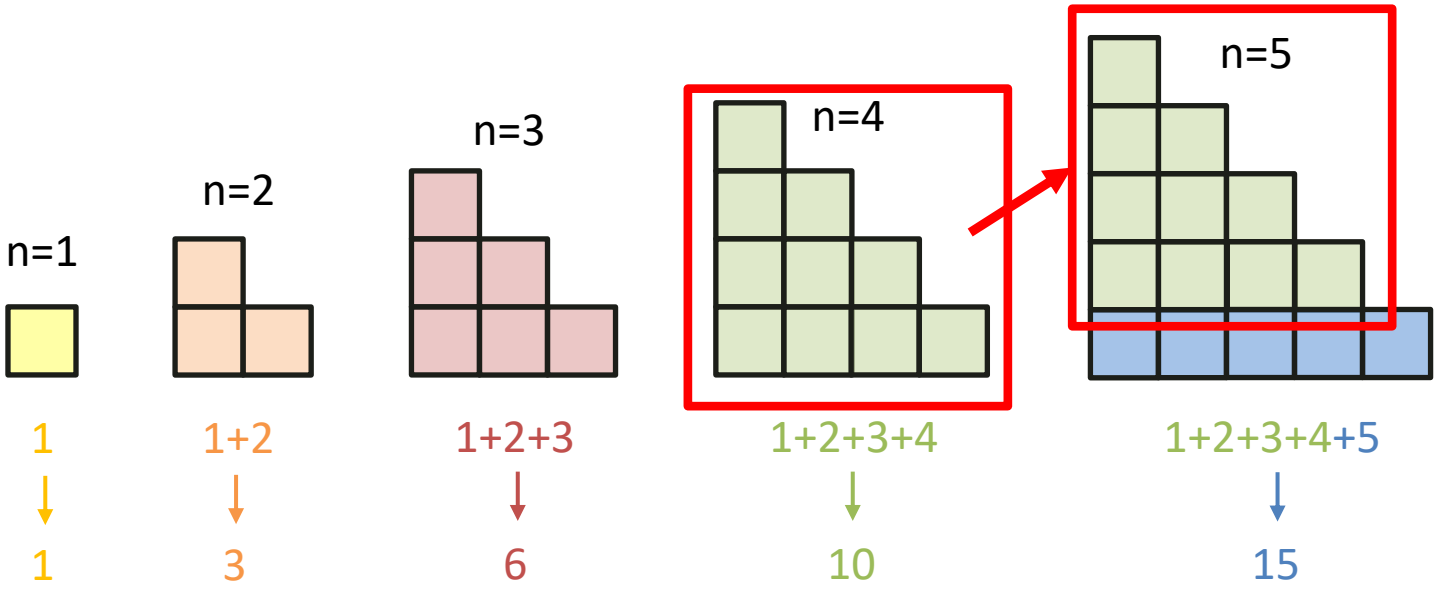
3. Erkenne den Zusammenhang

- „Wenn ich die Antwort für n=4 gegeben hätte, wie komme ich auf die Antwort für n=5? Für n=3 auf n=4? …“



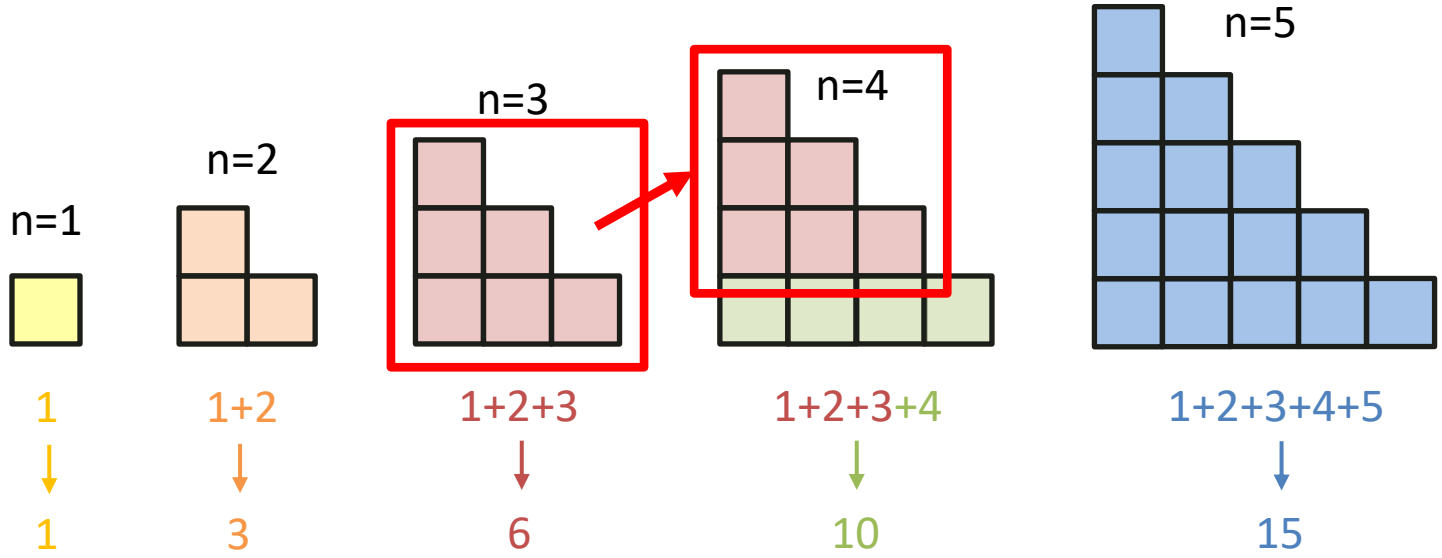
3. Erkenne den Zusammenhang

- „Wenn ich die Antwort für n=4 gegeben hätte, wie komme ich auf die Antwort für n=5? Für n=3 auf n=4? …“



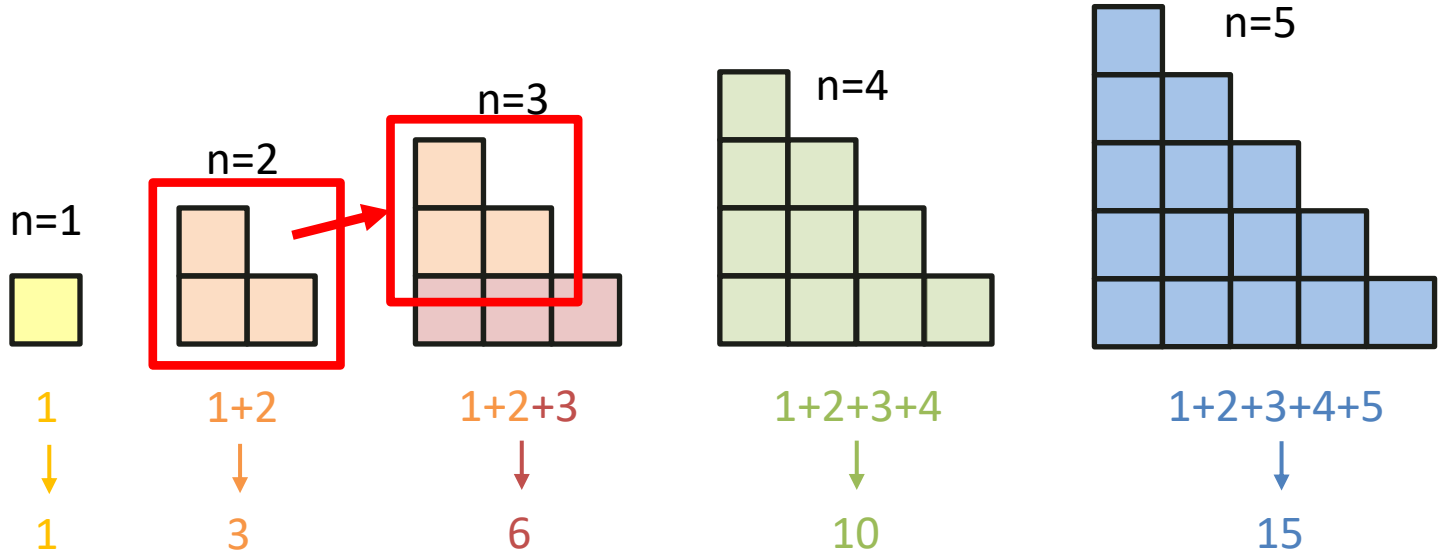
3. Erkenne den Zusammenhang

- „Wenn ich die Antwort für n=4 gegeben hätte, wie komme ich auf die Antwort für n=5? Für n=3 auf n=4? …“



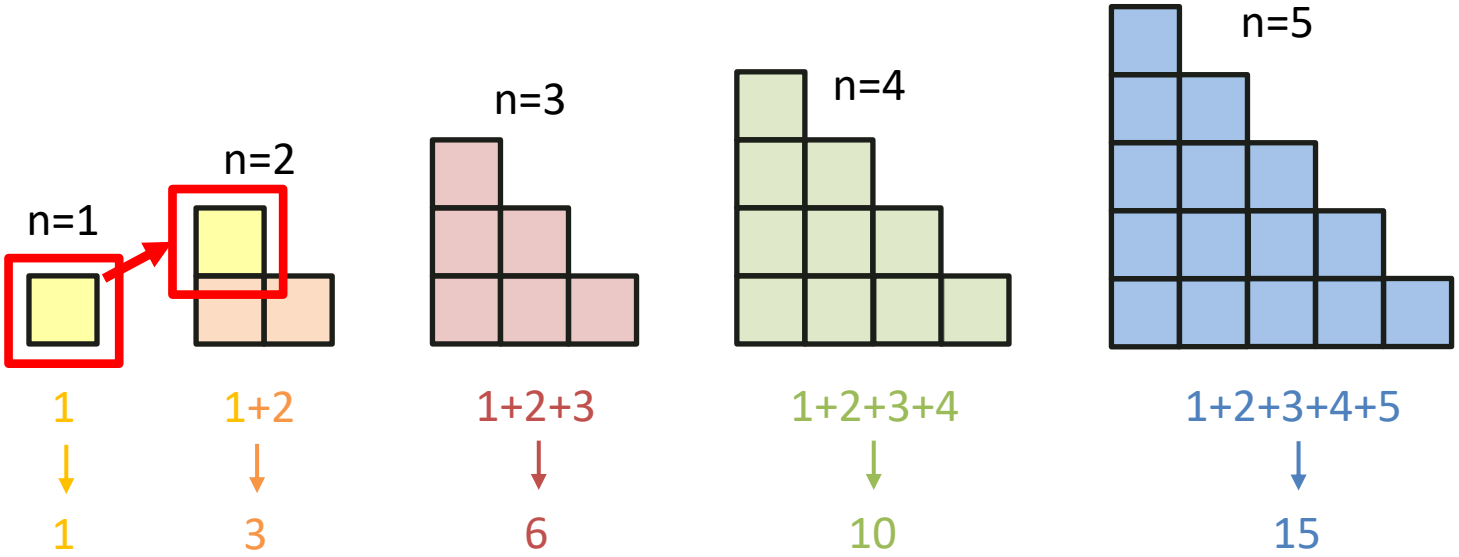
3. Erkenne den Zusammenhang

- „Wenn ich die Antwort für n=4 gegeben hätte, wie komme ich auf die Antwort für n=5? Für n=3 auf n=4? …“

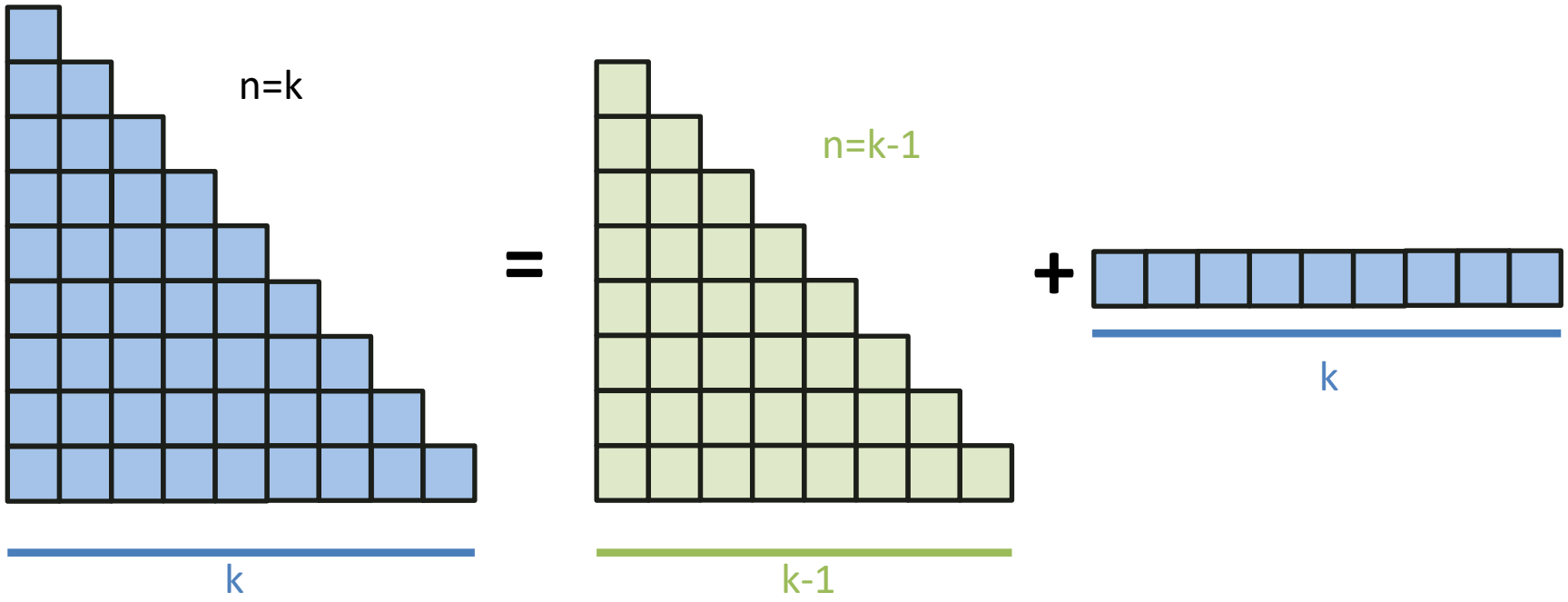


3. Erkenne den Zusammenhang

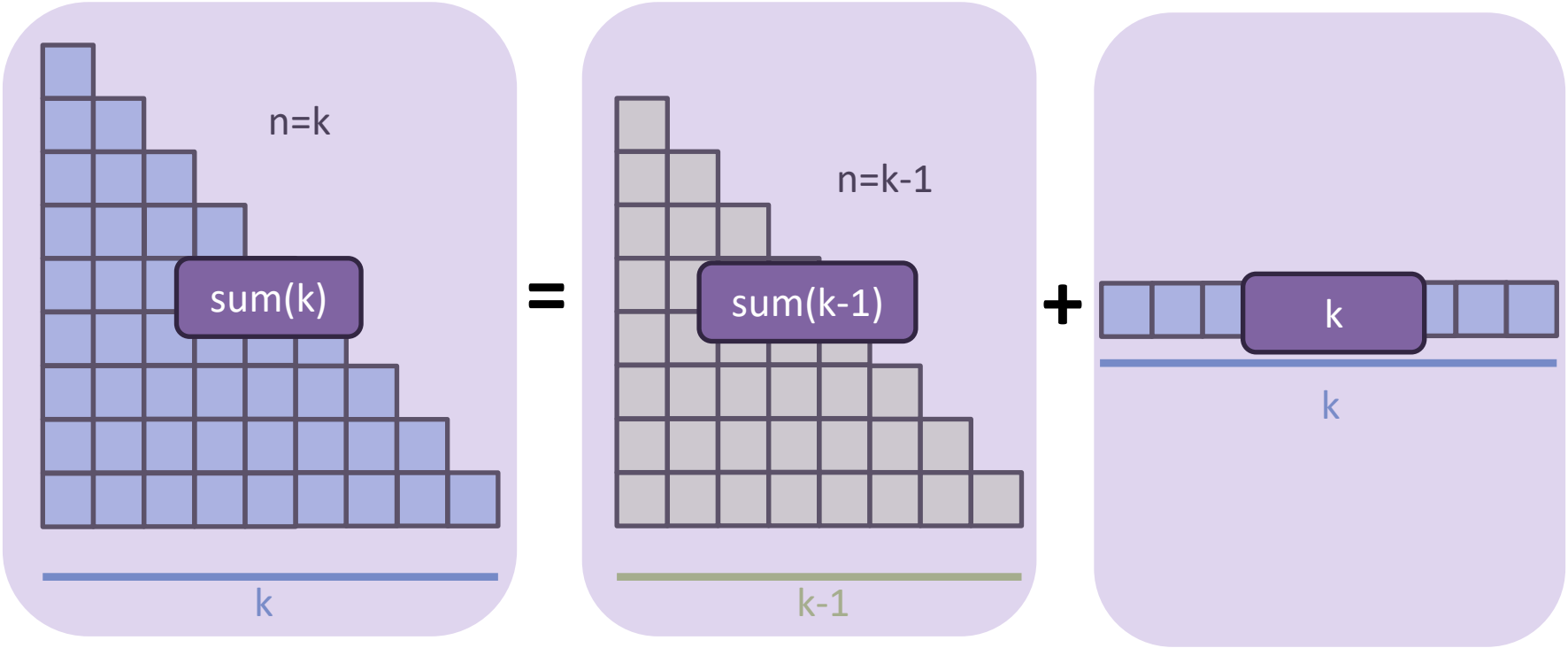
- „Wenn ich die Antwort für n=4 gegeben hätte, wie komme ich auf die Antwort für n=5? Für n=3 auf n=4? …“



4. Das Muster



4. Das Muster



Schritte

1. Was ist die einfachste mögliche Eingabe?

   → Base Case



2. Beispiele ausprobieren und visualisieren
3. Leite größere Beispiele von kleineren Beispielen ab
4. Verallgemeinere das Muster

   → Rekursionsschritt



5. Schreibe den Code

5. Der Code

Was haben wir herausgefunden?

Im 1. Schritt haben wir den *Base Case* gefunden:

Falls n=0, dann sum(n) = 0

Danach haben wir das *Rekursionsmuster* gefunden:

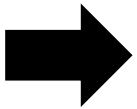
sum(n) = sum(n-1)+n

→ Dies können wir jetzt in Code umschreiben

5. Base Case + Muster → Code

Falls n=0: sum(n) = 0

Sonst: sum(n) = sum(n-1)+n



```
public static int sum(int n) {

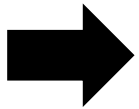


}
```

5. Base Case + Muster → Code

Falls n=0: sum(n) = 0

Sonst: sum(n) = sum(n-1)+n



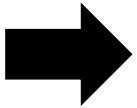
```
public static int sum(int n) {
    if (n == 0) {
        return 0;
    }

}
```

5. Base Case + Muster → Code

Falls n=0: sum(n) = 0

Sonst: sum(n) = sum(n-1)+n



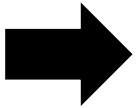
```
public static int sum(int n) {
    if (n == 0) {
        return 0;
    }

}
```

5. Base Case + Muster → Code

Falls n=0: sum(n) = 0

Sonst: sum(n) = sum(n-1)+n



```
public static int sum(int n) {
    if (n == 0) {
        return 0;
    }
    return sum(n-1) + n;
}
```

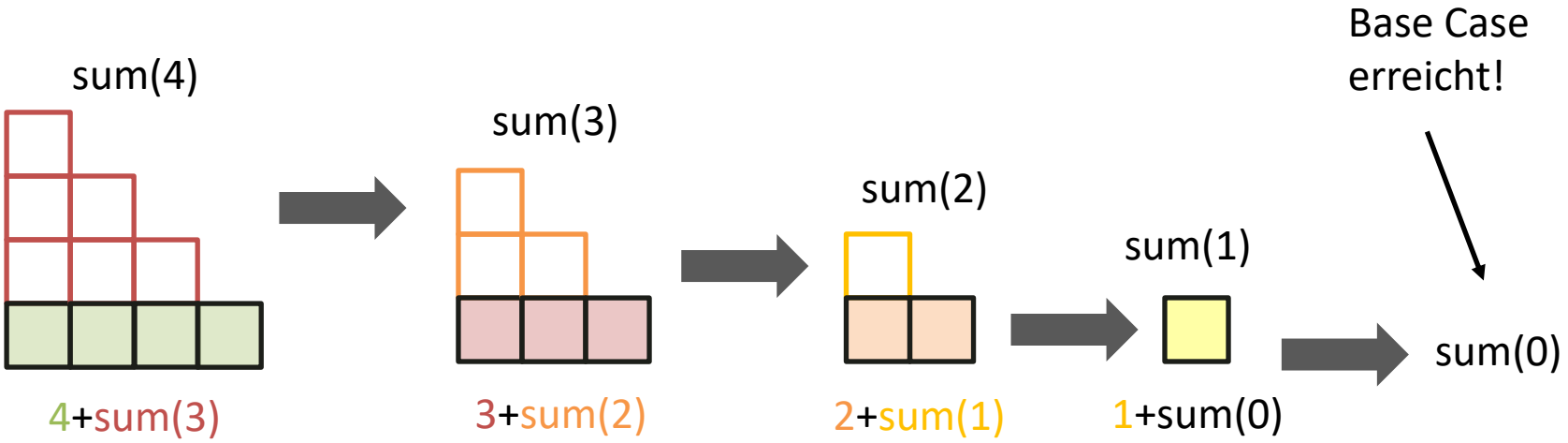
5. Base Case + Muster → Code

```
public static int sum(int n) {
    if (n == 0) {
        return 0;
    }
    return sum(n-1) + n;
}
```

Fragen?

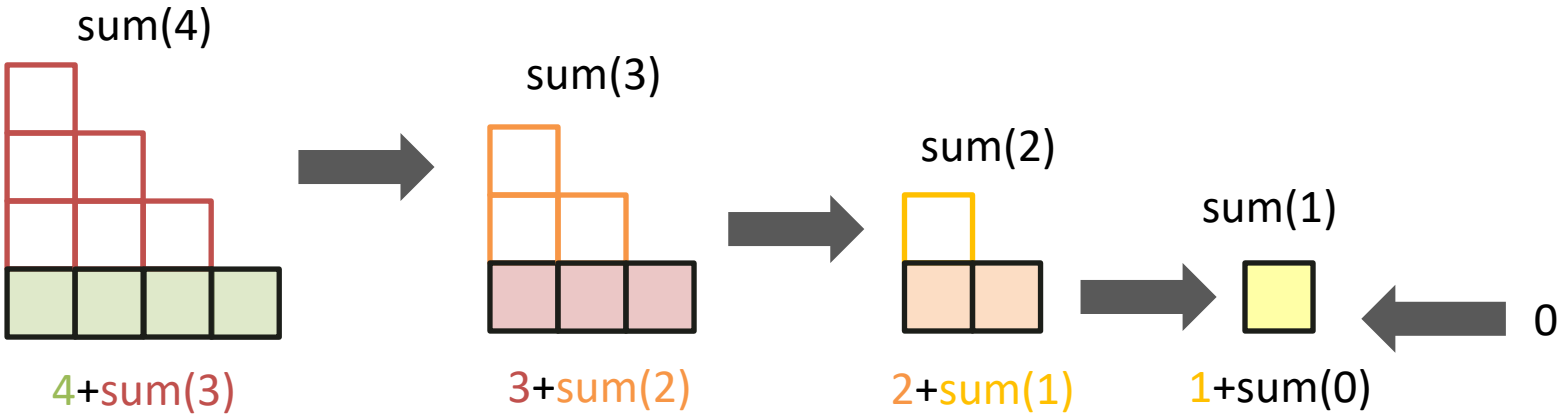
Was passiert wenn ich diesen Code ausführe?

1. Rekursive Aufrufe



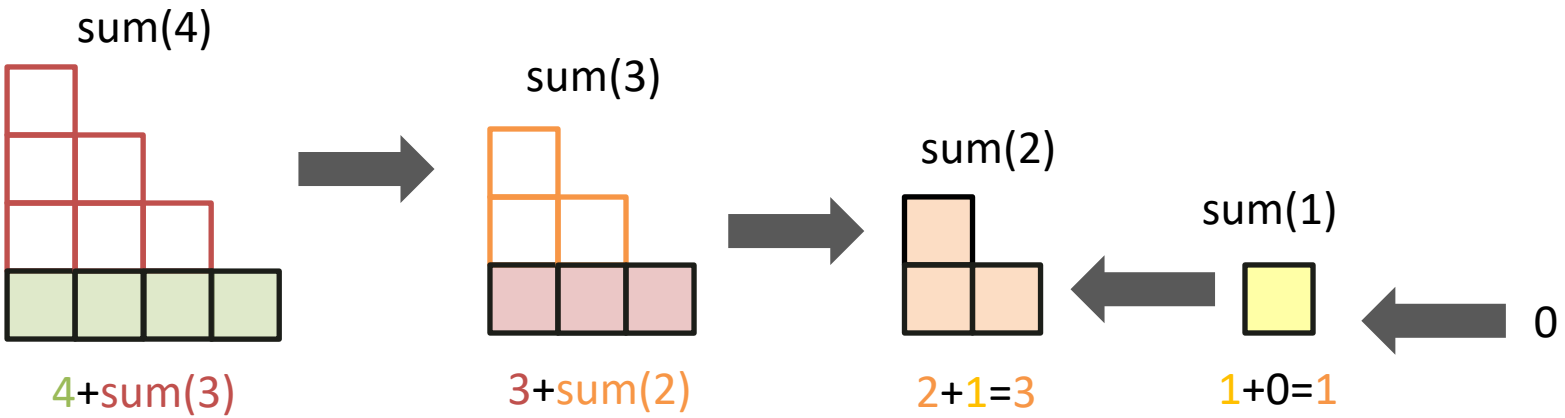
Was passiert wenn ich diesen Code ausführe?

2. Auflösen



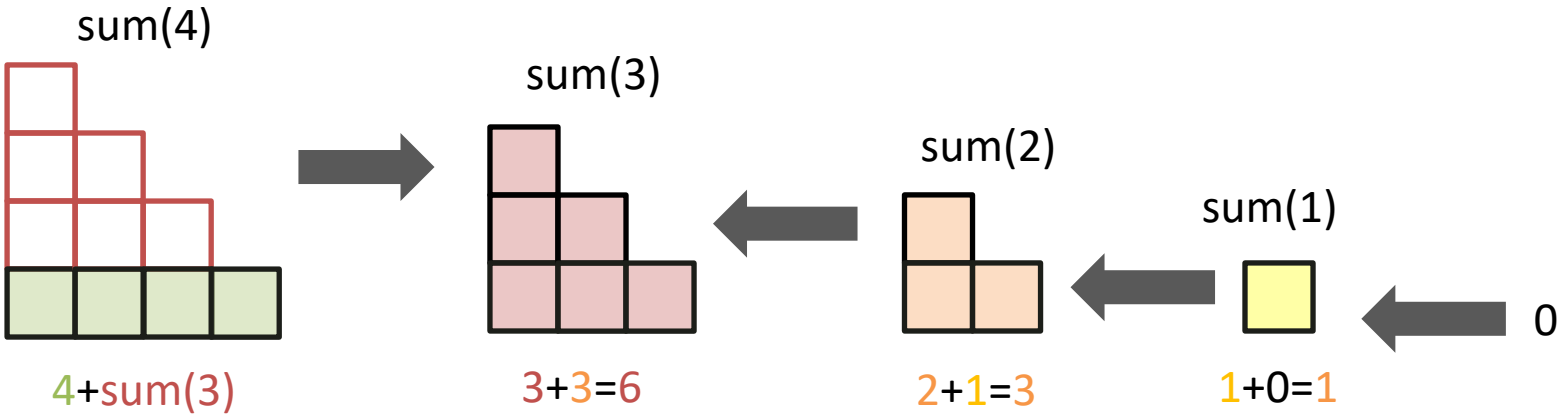
Was passiert wenn ich diesen Code ausführe?

2. Auflösen



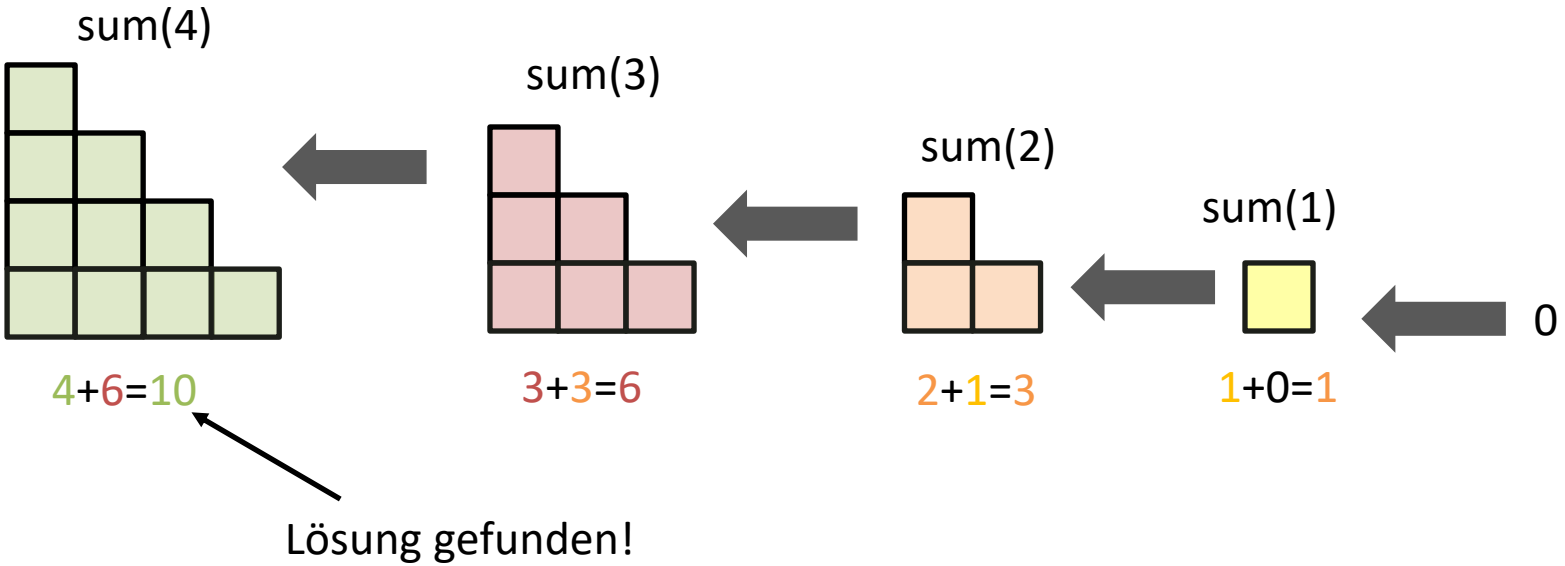
Was passiert wenn ich diesen Code ausführe?

2. Auflösen



Was passiert wenn ich diesen Code ausführe?

2. Auflösen



Rekursion in Java – Aufgabe

Erstelle eine Klasse `CheckIsPalindromRecursive`.
Implementiere eine **statische, rekursive Methode** `IsPalindrome`, die:

- Einen String akzeptiert
- Einen **boolean** zurückgibt, ob der gesamte String ein Palindrom ist.

Basisfall der Rekursion:



Rekursiver Fall:



Tipp: Benutze `str.charAt(i)`, `str.substring(i, j)`

Rekursion in Java – Aufgabe

Erstelle eine Klasse `CheckIsPalindromRecursive`.
Implementiere eine **statische, rekursive Methode** `IsPalindrome`, die:

- Einen String akzeptiert
- Einen **boolean** zurückgibt, ob der gesamte String ein Palindrom ist.

Basisfall der Rekursion:

- Wenn die String-Länge 0 oder 1 beträgt, gib `true` zurück
- Bei einer Länge von 2: Prüfe, ob beide Zeichen gleich sind, und gib entsprechend `true` oder `false` zurück.

Rekursiver Fall:



Tipp: Benutze `str.charAt(i)`, `str.substring(i, j)`

Rekursion in Java – Aufgabe

Erstelle eine Klasse `CheckIsPalindromRecursive`.
Implementiere eine **statische, rekursive Methode** `IsPalindrome`, die:

- Einen String akzeptiert
- Einen **boolean** zurückgibt, ob der gesamte String ein Palindrom ist.

Basisfall der Rekursion:

- Wenn die String-Länge 0 oder 1 beträgt, gib `true` zurück
- Bei einer Länge von 2: Prüfe, ob beide Zeichen gleich sind, und gib entsprechend `true` oder `false` zurück.

Rekursiver Fall:

- Nimm das **erste** und **letzte** Zeichen des Strings.
- Überprüfe, ob sie **gleich** sind.
- Verknüpfe das Ergebnis logisch mit einem Aufruf von `IsPalindrome` für den Teilstring ohne das erste und letzte Zeichen.

Tipp: Benutze `str.charAt(i)`, `str.substring(i, j)`

Rekursion in Java – Lösung

```java
public class Palindrome {
    public static boolean isPalindrome(String word) {
        if (word.length() <= 1)
            return true;
        boolean firstAndLast = word.charAt(0) == word.charAt(word.length() - 1);
        String withoutFirstAndLast = word.substring(1, word.length() - 1);
        return firstAndLast && isPalindrome(withoutFirstAndLast);
    }
}
```

Logisches Schliessen

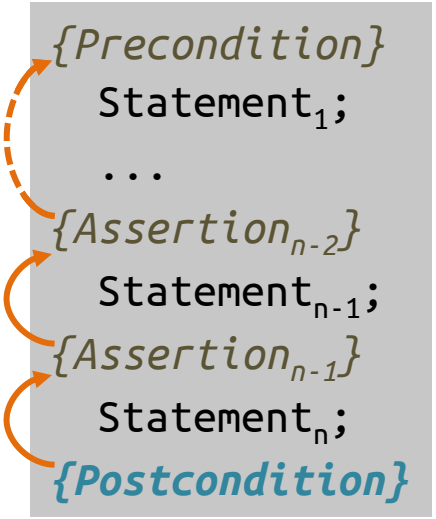
Hoare-Tripel: Syntax

- Ein **Hoare-Tripel** («**Hoare triple**»), auch *3-Tupel* genannt, besteht aus zwei Aussagen und einem Programmsegment:

```
{P} S {Q}
```

- Bestandteile eines Tripels
  - P ist die Vorbedingung (Precondition, z.B. $u > -59$)
  - S ist eine Anweisung (Statement; bzw. mehrere Anweisungen)
  - Q ist die Nachbedingung (Postcondition, z.B. $z > -59$)

Rückwärtsschliessen: Vorgehen



- **Start:** Wir beginnen bei der *Nachbedingung*
- **Schrittweise:** Herleiten der vorherigen Aussage (Assertion$_{i-1}$) durch Einbeziehen des *Effekts* der vorherigen Anweisung (Statement$_i$)
- **Ziel:** Herleiten einer *notwendigen und hinreichenden Vorbedingung*

- Rückwärts = «Welche Vorbedingung braucht mein Code, damit er die gewählten Garantien (Nachbedingung) geben kann?»

Typische Aufgaben:

- **Ist ein Hoare-Tripel gültig oder ungültig?**
  - Vor- und Nachbedingung gegeben
  - Folgt die Nachbedingung aus dem Code und der Vorbedingung?
- **Was ist die schwächste Precondition?**
  - Postcondition ist gegeben
  - Welche Werte erfüllen unsere Postcondition?

Stärkere und schwächere Aussagen

- Wir können *stärkere* und *schwächere* Aussagen unterscheiden
  - Wenn $P_1 \Longrightarrow P_2$ gilt, dann ist P_1 stärker als P_2 (und P_2 schwächer als P_1)
  - $\Longrightarrow$ bedeutet: Falls P_1 wahr ist, ist P_2 auch wahr.
  - Bsp: P_1: $x > 4$ (Stärker) P_2: $x > 0$ (Schwächer)
- **Starke Aussage:** Eine Aussage oder Voraussetzung, die **mehr Einschränkungen** auferlegt. Sie gilt in weniger Fällen.
- **Schwache Aussage**: Eine Aussage oder Voraussetzung, die **weniger Einschränkungen** auferlegt. Sie ist allgemeiner.

Preconditions / Postconditions

- Die **schwächste Precondition** ist die schwächste Precondition, für die noch die Postcondition erfüllt ist.
  - «Was dürfen wir der Funktion für Werte geben, damit wir einen erwünschten Output haben?»
  - Wir wollen viele mögliche Werte zulassen
- Die **stärkste Postcondition** ist die stärkste Postcondition, welche noch aus der Precondition folgt.
  - «Was kann alles als Resultat kommen?»
  - Wir wollen einschränken, was für Werte wir bekommen können

Weakest Precondition – Einfaches Beispiel

{ }

x = y*y ———————

{x>4}

Weakest Precondition – Einfaches Beispiel

{ }

$x = y*y$ ——— $\{x>4\}$

Weakest Precondition – Einfaches Beispiel

{ }

$x = y*y$ ——— $\{y*y>4\}$

Weakest Precondition – Einfaches Beispiel

{y*y>4}

x = y*y ————

Weakest Precondition – Einfaches Beispiel

{|y|>2}

x = y*y ————

Weakest Precondition – Zweites Beispiel

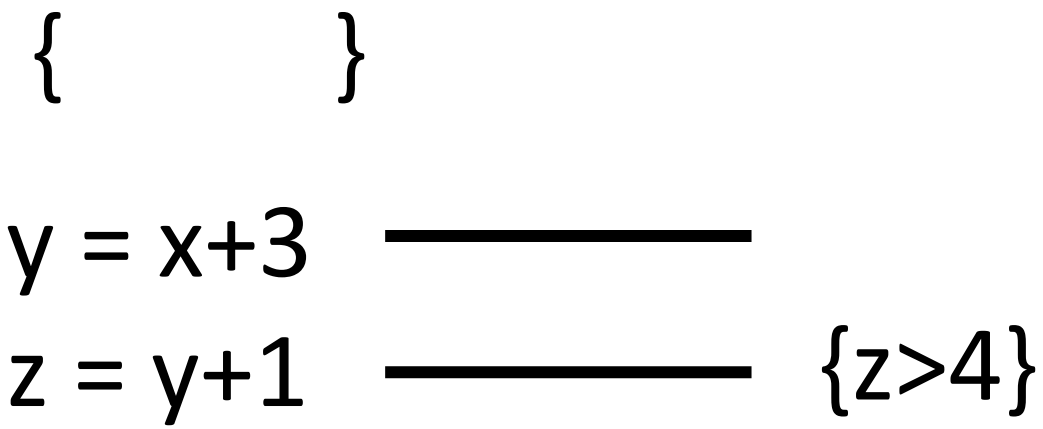
{ }

$y = x+3$ ———

$z = y+1$ ———

{z>4}

Weakest Precondition – Zweites Beispiel



Weakest Precondition – Zweites Beispiel

{ }

y = x+3 ———

z = y+1 ——— {y+1 >4}

Weakest Precondition – Zweites Beispiel

{ }

y = x+3 ——— {y+1 >4}

z = y+1 ———

Weakest Precondition – Zweites Beispiel

{ }

$y = x+3$ ——— $\{x+3+1 > 4\}$

$z = y+1$ ———

Weakest Precondition – Zweites Beispiel

{ }

y = x+3 ——— {x >0}

z = y+1 ———

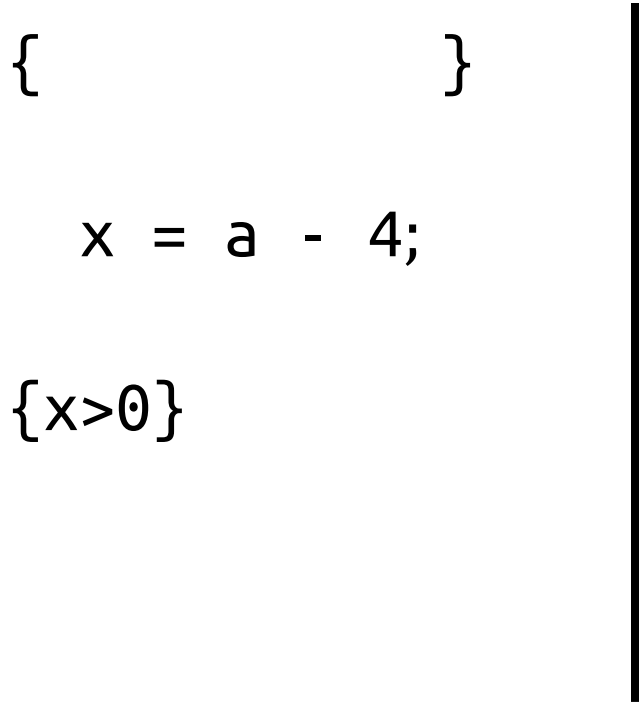
Weakest Precondition – Zweites Beispiel

$\{x > 0\}$

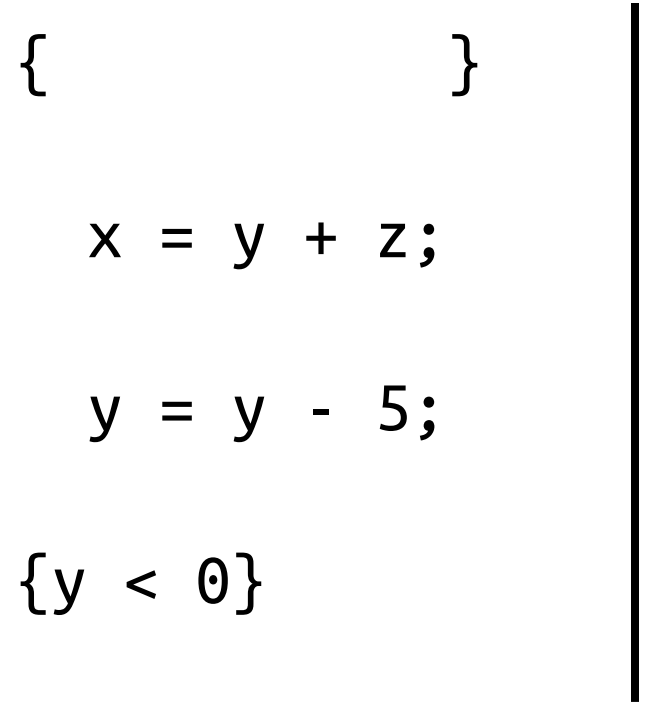
$y = x+3$ ——————

$z = y+1$ ——————

Schwächste Vorbedingung - Beispiel



Schwächste Vorbedingung - Beispiel



Kahoot

FS 21 Prüfungsaufgabe

Bitte geben Sie für die folgenden Programmsegmente die schwächste Vorbedingung (weakest precondition) an. Bitte verwenden Sie Java Syntax (die Klammern `{` und `}` können Sie weglassen). Alle Anweisungen sind Teil einer Java Methode. Alle Variablen sind vom Typ `int` und es gibt keinen Overflow.

1. `P: ??`

```
m = n * 4;  k = m - 2;
Q: { n > 0 && k > 5 }
```

2. `P: ??`

```
if (x > y) {
  z = x - y;
} else {
  z = y - x;
}
{ z > 0 }
```

Vorbesprechung

Aufgabe 1: Präfixkonstruktion



Gegeben seien zwei Strings `s` und `t` und ein Integer `n` mit $n \geq 0$. Schreiben Sie ein Programm, das zurückgibt, ob `s` eine Konkatenation von maximal `n` vielen Präfixen von `t` ist.

Beispiele:

- `s = "abcababc"`, `t = "abc"`, `n = 4`: Das Programm sollte `true` zurückgeben, da `"abc"` und `"ab"` Präfixe von `t` sind und `s` eine Konkatenation von `"abc"`, `"ab"`, `"abc"` ist.
- `s = "abcbcabc"`, `t = "abc"`, `n = 4`: Das Programm sollte `false` zurückgeben, da `"bc"` kein Präfix von `t` ist.
- `s = "abab"`, `t = "abac"`, `n = 2`: Das Programm sollte `true` zurückgeben, da `"ab"` ein Präfix von `t` ist und `s` eine Konkatenation von `"ab"`, `"ab"` ist.

Implementieren Sie die Methode `isPrefixConstruction(String s, String t, int n)` in der Klasse `PrefixConstruction`. Die Methode hat drei Argumente: die beiden Strings `s` und `t` und der Integer `n`. Sie dürfen davon ausgehen, dass der Integer grösser oder gleich 0 ist. In der Datei "PrefixConstructionTest.java" finden Sie Tests.

Tipp: Lösen Sie die Aufgabe rekursiv.

Tipp: Prüfe s = "ababa" t = "aba" n = 2 auf Papier.

Aufgabe 2: Weakest Precondition



Bitte geben Sie für die folgenden Programmsegmente die schwächste Vorbedingung (weakest precondition) an. Bitte verwenden Sie Java-Syntax. Alle Anweisungen sind Teil einer Java Methode. Alle Variablen sind vom Typ `int` und es gibt keinen Overflow.

1.
```
P: { ?? }
S:   m = n * 4;  k = m - 2;
Q: { n > 0 && k > 5 }
```

2.
```
P: { ?? }
S:   m = n * n;  k = m * 2;
Q: { k > 0 }
```

3.
```
P: { ?? }
S:   y = x + 3; z = y + 1;
Q: { z > 4 }
```

4.
```
P: { ?? }
S:   y = x + 1; z = y - 3;
Q: { z == 10 }
```

Beispiel

```
P: { ?? }
S:   a = b * 3; c = a + 1;
Q: { a > 0 && c < 5 }
```

Aufgabe 3: Wörter Raten



Das Programm "WoerterRaten.java" enthält Fragmente eines Rate-Spiels, welches Sie vervollständigen sollen. In dem Spiel wählt der Computer zufällig ein Wort w aus einer Liste aus und der Mensch muss versuchen, das Wort zu erraten. In jeder Runde kann der Mensch eine Zeichenfolge z (welche einen oder mehrere Buchstaben enthält) eingeben und der Computer gibt einen Hinweis dazu. Folgende Hinweise sind möglich:

1. w beginnt mit z
2. w endet mit z
3. w enthält z
4. w enthält nicht z

```
Tipp? e
Das Wort enthält nicht "e"!
Tipp? a
Das Wort endet mit "a"!
Tipp? j
Das Wort beginnt mit "j"!
Tipp? v
Das Wort enthält "v"!
Tipp? java
Das Wort ist "java"!
Glückwunsch, du hast nur 5 Versuche benötigt!
```

Aus Textdatei lesen

woerter.txt

```
8
wort
blasinstrument
computer
schlange
java
programmieren
welt
sugus
```

Code zum Einlesen der Wörter:

```
Scanner scanner = new Scanner(new File("woerter.txt"));
String[] woerter = new String[scanner.nextInt()];
for(int i = 0; i < woerter.length; i++) {
    woerter[i] = scanner.next();
}
```

Anzahl Wörter = 8

Liest das nächste Wort

Absoluter Pfad

- beginnt mit `C:\...` unter Windows oder `/...` unter Linux/macOS

Relativer Pfad

- ist relativ zum aktuellen Verzeichnis (“working directory”) des Programms (bei uns der Projektordner)

Aufgabe 4: Datenanalyse



In dieser Aufgabe werden Sie die Kelchblattlänge von Iris Blumen, welche auch Schwertlilien genannt werden, analysieren. Dazu verwenden Sie ein öffentlich zugängliches Dataset [1] welches die Längen vom Kelchblatt (Sepal Length) von jeweils 150 verschiedenen Iris Blumen enthält. Es gibt sehr viele Arten von Iris Blumen, insgesamt sind 285 Arten bekannt. Diese zu unterscheiden ist ein komplexer Prozess. Wir werden jedoch versuchen mittels der Länge des Kelchblattes drei Arten von Iris Blumen zu unterscheiden, in dem wir die gegebenen Daten analysieren. Wir werden uns auf die folgenden drei Iris Blumen konzentrieren:

- Iris setosa, auch Borsten-Schwertlilie genannt.
- Iris versicolor, auch Verschiedenfarbige Schwertlilie gennannt.
- Iris virginica, auch blaue Sumpfschwertlilie genannt.



Iris Versicolor **Iris Setosa** **Iris Virginica**

Aufgabe 4: Datenanalyse

3. Die drei Werte, die Sie in 2. berechnet haben, sagen nicht viel über die Daten aus. Um die Daten besser zu verstehen, soll Ihr Programm ein Histogramm berechnen und auf der Konsole ausgeben. Die Textausgabe könnte ungefähr so aussehen:

```
[40,43)
[43,46)
[46,49)
[49,52) |
[52,55)
[55,58) ||
[58,61) ||||||
[61,64) ||||||||||
[64,67) |||||||||
[67,70) ||||||||||
[70,73) ||||
[73,76) ||
[76,79) |||||
[79,82) |
[82,85)
```

Implementieren Sie die Methode `histogrammAnalyse()`. Diese Methode soll zuerst den Benutzer nach der Anzahl der Histogramm-Klassen fragen, dann das Histogramm berechnen und schliesslich ausgeben. Zwei (leere) Methoden sind schon vorgegeben: `erstelleHistogramm()` und `klassenBreite()`. Für diese beiden Methoden existieren Tests in "DatenAnalyseTest.java" und Kommentare, welche Ihnen beim Schreiben des Programms helfen. Überlegen Sie sich, wie Sie das Problem weiter aufteilen möchten, und erstellen Sie entsprechende Methoden.

Bonusaufgabe



Aufgabe 5: Matrix (Bonus!)

Achtung: Diese Aufgabe gibt Bonuspunkte (siehe "Leistungskontrolle" im `www.vvz.ethz.ch`). Die Aufgabe muss eigenhändig und alleine gelöst werden. Die Abgabe erfolgt wie gewohnt per Push in Ihr Git-Repository auf dem ETH-Server. Verbindlich ist der letzte Push vor dem Abgabetermin. Auch wenn Sie vor der Deadline committen, aber nach der Deadline pushen, gilt dies als eine zu späte Abgabe. Bitte lesen Sie zusätzlich die allgemeinen Regeln.